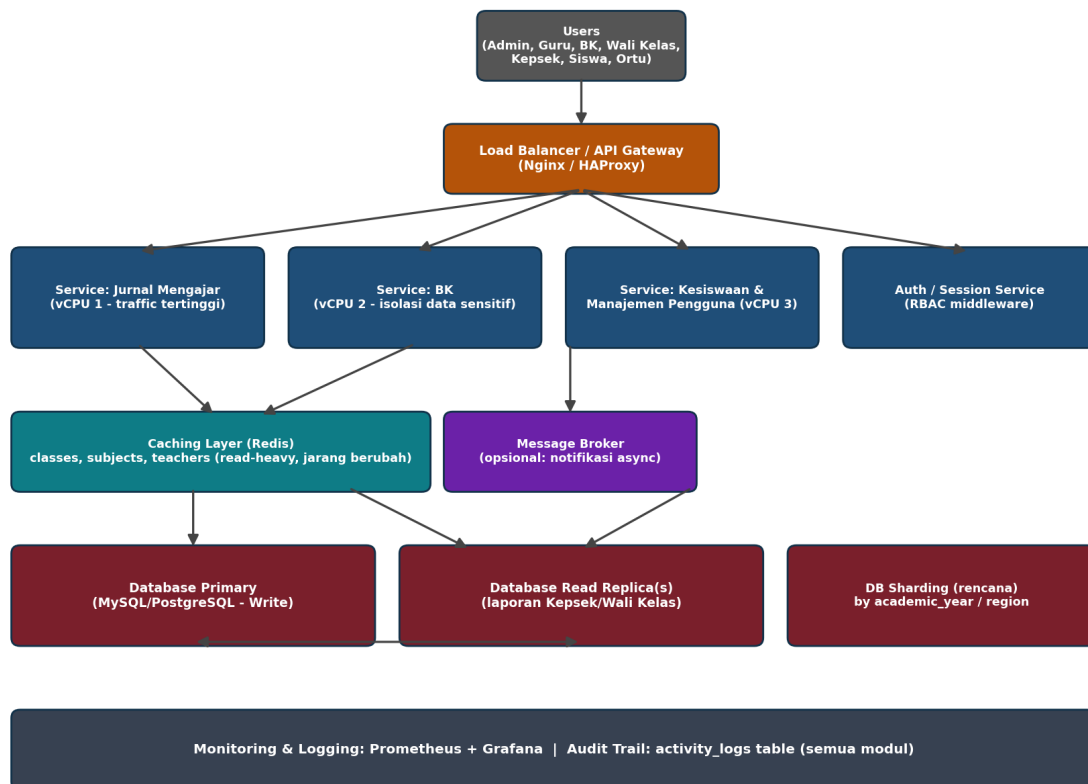


SOFTWARE DESIGN DOCUMENT (SDD)

SISTER — Sistem Informasi Sekolah Terintegrasi Berbasis Web
Mata Kuliah: Advanced Software Testing & Quality Assurance (ASTQA)

1. High-Level Architecture Diagram

Diagram berikut menggambarkan target arsitektur scalable untuk SISTER: satu Load Balancer/API Gateway di depan, layanan dipisah per beban kerja (Jurnal Mengajar, BK, Kesiswaan+Manajemen Pengguna, Auth), caching layer (Redis) untuk data referensi yang jarang berubah, database primary dengan read-replica untuk laporan, rencana sharding, serta monitoring & logging terpusat.



Catatan implementasi saat ini (baseline MID): sistem berjalan sebagai modular monolith satu proses Node.js dengan SQLite. Load balancer, caching, replikasi, dan sharding pada diagram di atas merupakan RENCANA scalability (bagian dari kebutuhan non-fungsional) yang belum diimplementasikan secara fisik — lihat gap analysis pada README repositori final project.

2. Detail Modul & Alur Interaksi Antarmodul

- Auth Module (routes/auth.js + lib/auth.js): menangani login/logout, membuat session, dan menyediakan middleware requireAuth/requireRole yang dipakai oleh SELURUH modul lain sebelum route handler dieksekusi.
- Users Module (routes/users.js): CRUD user, reset/aktivasi akun; menulis ke tabel users & activity_logs.
- Students Module (routes/students.js): CRUD siswa & kelas; dibaca oleh Journal & BK module melalui foreign key student_id/class_id (data tidak diduplikasi antarmodul).
- Journal Module (routes/journal.js): mencatat jurnal mengajar; terhubung ke teachers, classes, subjects sekaligus.
- BK Module (routes/bk.js): mencatat kasus BK, pelanggaran, prestasi, dan catatan konseling; seluruhnya mereferensikan students.id sehingga riwayat siswa dapat direkap lintas modul.

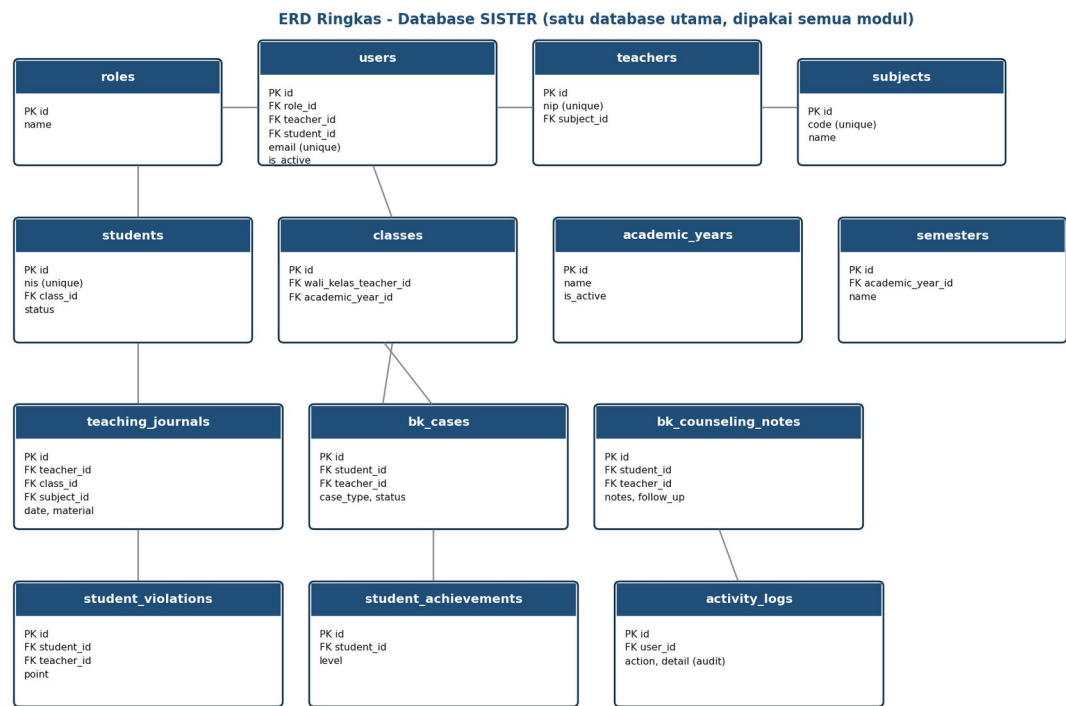
Seluruh modul berkomunikasi dengan database melalui helper get/all/run pada db/index.js — abstraksi ini memungkinkan migrasi ke REST/gRPC antar-service tanpa mengubah logika bisnis di routes/*.js apabila monolith dipecah menjadi microservices di kemudian hari.

3. API Contract (Endpoint Utama)

Method	Endpoint	Request Payload	Response / Status Code
POST	/login	email, password (form-urlencoded)	302 → /dashboard (sukses) atau 302 → /login (gagal, flash error)
POST	/logout	-	302 → /login
GET	/dashboard	-	200 OK (HTML, statistik sesuai role) / 302 → /login jika belum auth
GET	/users	-	200 OK (admin only) / 403 role lain
POST	/users/create	name, email, password, role_id, teacher_id?, student_id?	302 → /users; error jika email duplikat
POST	/users/:id/toggle	-	302 → /users (toggle is_active)
GET	/students	query: class (khusus wali kelas)	200 OK (list terfilter sesuai role) / 403
POST	/students/create	nis, name, class_id, gender, birth_date, parent_name, status	302 → /students; error jika NIS duplikat (unique constraint)
GET	/students/export	-	200 OK, Content-Type: text/csv (admin only)
POST	/students/:id/delete	-	302 → /students; gagal jika ada relasi FK aktif (jurnal/BK)
GET	/journal	query: class (wali kelas)	200 OK (terfilter per role) / 403
POST	/journal/create	class_id, subject_id, date, material, method, notes	302 → /journal (guru only)
POST	/journal/:id/update	idem create	302 → /journal; 403 jika bukan pemilik jurnal
GET	/bk	-	200 OK (admin, guru_bk, wali_kelas, kepala_sekolah) / 403 guru biasa
POST	/bk/create	student_id, case_type, date, description, status	302 → /bk (guru_bk only)
POST	/bk/:id/notes	date, notes, follow_up	302 → /bk/:id (guru_bk only)
GET	/logs	-	200 OK, audit trail 200 entri terbaru (admin only)

4. Database Schema (ERD)

Satu database utama dipakai bersama seluruh modul. Relasi kunci: users.role_id → roles.id (dasar RBAC); students.class_id → classes.id; teaching_journals menghubungkan teachers, classes, dan subjects sekaligus; bk_cases/bk_counseling_notes/student_violations/student_achievements seluruhnya mereferensikan students.id.



Optimasi yang diterapkan: index pada kolom foreign key transaksional (class_id, teacher_id, student_id), unique constraint pada email dan nis, serta foreign_keys=ON untuk menjaga konsistensi data lintas modul.

5. Strategi Scalability

Aspek	Strategi
Horizontal Scaling	Modul dengan beban tertinggi (Jurnal Mengajar, diakses semua guru tiap hari) digandakan menjadi beberapa instance di belakang load balancer.
Vertical Scaling	Database server ditingkatkan spesifikasi CPU/RAM saat volume data (siswa, jurnal, kasus BK) bertambah besar.
Caching	Data referensi yang jarang berubah (daftar kelas, mapel, guru) di-cache di Redis agar tidak selalu query ke database.
DB Replication	Read-replica dipakai khusus untuk query laporan/rekap (Kepala Sekolah, Wali Kelas) agar tidak membebani database primary (write).
DB Sharding (rencana)	Data dapat di-shard berdasarkan academic_year atau region/cabang sekolah jika skala multi-sekolah diterapkan.
Session Store	Session yang saat ini in-memory (satu proses) dipindahkan ke Redis agar tidak menjadi single point of failure saat multi-instance.